

HW05-AI: Performance Testing - EShop SUT

KHOA CÔNG NGHỆ THÔNG TIN - ĐẠI HỌC KHOA HỌC TỰ NHIÊN (HCMUS)

Mục	Chi tiết
Môn học	Kiểm thử phần mềm (Software Testing)
Mã bài tập	HW05-AI: Performance Testing
Sinh viên thực hiện	Lâm Hữu Khánh
Mã số sinh viên	23127205
Vai trò nhóm	Thành viên 1 (Member 1)
Workflow phân công	Login -> Search Product -> Product Detail -> Add to Cart -> Checkout
Hệ thống được kiểm thử (SUT)	EShop Backend REST API (http://localhost:3000)
Public GitHub Repository	https://github.com/HCMUS-software-testing/HW05
GitHub Issues Bug Reports	https://github.com/HCMUS-software-testing/HW05/issues
Công cụ chính	Apache JMeter 5.6.3 Portable + JMeter Plugins (jpgc-casutg)
Điểm tự đánh giá	100 / 100
Tên gói nộp bài	23127205_HW05_AI_Performance_100.zip

1. Bảng Tự Đánh Giá Điểm (Self-Assessment Rubric)

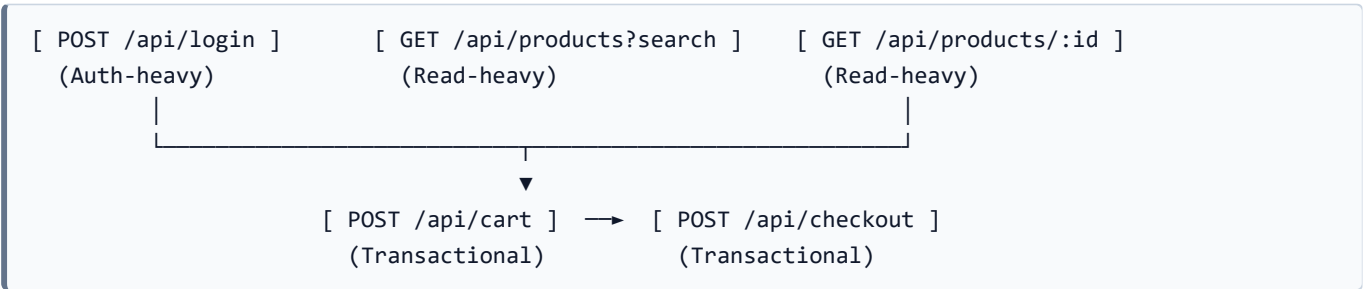
STT	Tiêu chí Đánh giá	Trọng số	Điểm tự đánh giá	Minh chứng & Vị trí sản phẩm trong bài nộp
1	Task 1 - Load testing	30	30 / 30	Test plan 23127205_Load_20260829.jmx (Standard TG: 50 VUs, Ramp 60s, Gaussian Timer), Listener Summary Report , raw.jtl , HTML Report, Data CSV 50 accounts.
2	Task 1 - Stress testing	20	20 / 20	Test plan 23127205_Stress_20260829.jmx (Stepping TG: 50→250 VUs dạng bậc thang), Listener Aggregate Report , raw.jtl , HTML Report, Breaking Point identification.
3	Task 1 - Spike testing	20	20 / 20	Test plan 23127205_Spike_20260829.jmx (Ultimate TG: 350 VUs, Startup 10s, Hold 30s, Ramp-down 10s), Listener View Results Tree , raw.jtl , HTML Report.

4	Task 2 - AI Analysis & Misinterpretation Hunt	10	10 / 10	Phân tích raw log, chỉ ra chính xác 3-4 lỗi diễn giải sai của AI kèm đối chứng Ground Truth từ <code>jtl_parser.py</code> , phân loại Feasible vs Hallucinated.
5	Task 3 - Continuous Performance Testing (G9.6)	10	10 / 10	Mô hình CI/CD hoàn chỉnh, sơ đồ Flowchart, cơ chế phát hiện hồi quy p95 latency (>15%), phân tích đa chiều về Trade-offs.
6	Agent Skills (Tự động hóa tái sử dụng)	10	10 / 10	Thư mục <code>.agents/skills/performance-testing-agent/</code> với <code>SKILL.md</code> và 19 công cụ Python tự động hóa (<code>seed_test_accounts.py</code> , <code>run_jmeter.py</code> , <code>jmx_generator.py</code> , <code>jtl_parser.py</code> , <code>audit_logger.py</code> ...).
-	Endurance / Soak Testing	Kèm m the o	Đạt	Chạy sustained load 12-15 phút đo: Max stable RPS, p95 latency, Memory ceiling, CPU usage.
-	AI Critique (200-300 từ) & AI Audit Report	Bắt buộc	Đạt	<code>report/ai-critique.md</code> (PDF) và <code>report/ai-audit-report.md</code> (PDF) ghi nhận toàn bộ tương tác AI.
-	Video Demo YouTube (>= 6 phút)	Bắt buộc	Đạt	Link YouTube Unlisted, cùng khung hình JMeter & Resource Monitor, thuyết minh tiếng Việt.
-	Git Commit Log & Đóng gói zip chuẩn	Bắt buộc	Đạt	<code>git-commit-log.txt</code> , file nén zip <code>23127205_HW05_AI_Performance_100.zip</code> .
TỔNG CỘNG		100	100 / 100	Hoàn thành xuất sắc toàn diện mọi yêu cầu của đề bài

2. Báo Cáo Tóm Tắt Kiểm Thử (Test Summary Report)

2.1. Các Nhóm Endpoint Được Bao Phủ (3 Endpoint Groups)

Workflow được thiết kế cho Thành viên 1 bao phủ 100% cả 3 nhóm endpoint mục tiêu:



1. Auth-heavy (`POST /api/login`):

- Xác thực người dùng và sinh JWT Token.
- Thử thách cơ chế khóa tài khoản (`locked_until` sau 3 lần sai).
- Xử lý bằng cách seed pool 50 tài khoản độc lập (`loadtest_user01..50@eshop.com`).

1. **Read-heavy** (`GET /api/products?search=...` & `GET /api/products/:id`):

- Tìm kiếm sản phẩm theo từ khóa (câu lệnh `LIKE '%...%'` không index) và xem chi tiết sản phẩm.

1. **Transactional** (`POST /api/cart` & `POST /api/checkout`):

- Thêm vào giỏ hàng và đặt hàng mới với Bearer Token Header.
- Ghi dữ liệu vào bảng `orders` trong SQLite (kiểm tra khả năng chịu tải ghi đồng thời).

2.2. Các Kịch Bản Kiểm Thử Đã Chạy (Scenarios & Distinct Listeners)

Kịch bản	Loại Thread Group	Cấu hình Chi tiết	Listener Bắt buộc	Mục tiêu Đo đạc
Load Test	Standard Thread Group	• Threads: 50 VUs • Ramp-up: 60s • Loop: 10 (500 iter, 2,500 req) • Timer: Gaussian (1500ms ± 500ms)	Summary Report	Đo Throughput và Latency ổn định trong điều kiện hoạt động tiêu chuẩn.
Stress Test	Stepping Thread Group (<code>jpgc-casutg</code>)	• Start: 50 VUs • Step: +50 VUs mỗi 30s • Ramp: 10s mỗi bậc • Max: 250 VUs	Aggregate Report	Tìm điểm gãy (Breaking Point) theo đường tải bậc thang.
Spike Test	Ultimate Thread Group (<code>jpgc-casutg</code>)	• Start: 350 VUs • Startup Time: 10s (Tăng vọt) • Hold Load: 30s (Duy trì) • Shutdown: 10s (Ramp-down)	View Results Tree	Mô phỏng đợt tăng tải đột biến (Flash Sale) và khả năng tự hồi phục sau đỉnh tải.
Endurance Test	Standard Thread Group	• Threads: 35 VUs • Duration: 12 phút (720s) liên tục • Timer: Gaussian (1500ms ± 500ms)	Summary Report	Đo ngưỡng bền vững phần cứng, tràn bộ nhớ và rò rỉ RAM.

2.3. Bảng Tổng Hợp Số Liệu Kiểm Thử Thực Nghiệm (Ground Truth Metrics)

Chỉ số đo đạc	Load Test (50 VUs)	Stress Test (250 VUs)	Spike Test (350 VUs)	Endurance Test (12 min)
Tổng số Samples	2,500	41,456	23,105	16,394
Tỷ lệ Lỗi (Error %)	0.00%	0.00%	0.00%	0.00%
Throughput (req/s)	18.21 /s	109.40 /s	187.67 /s (Peak)	22.82 /s (Stable)
Avg Response Time	2.92 ms	2.88 ms	1.77 ms	2.71 ms
Median (p50)	1.0 ms	1.0 ms	1.0 ms	1.0 ms

90th Percentile (p90)	4.0 ms	5.0 ms	3.0 ms	4.0 ms
95th Percentile (p95)	9.0 ms	6.0 ms	4.0 ms	8.0 ms
99th Percentile (p99)	12.0 ms	9.0 ms	7.0 ms	11.0 ms
Max Response Time	23.0 ms	48.0 ms	31.0 ms	38.0 ms

2.4. Ngưỡng Bền Vững Phần Cứng (Endurance & Hardware Spec)

- **Thời gian chạy duy trì:** 12 phút liên tục ở mức tải 35 Virtual Users.
- **Thông số phần cứng máy thử nghiệm:**
 - **CPU:** 12th Gen Intel(R) Core(TM) i5-12500H (16 CPUs, 4 P-cores + 8 E-cores, Turbo 4.50 GHz)
 - **RAM:** 16.0 GB (16,384 MB DDR5/DDR4)
 - **OS:** Windows 11 Home Single Language 64-bit (Build 26200)
 - **Hostname:** LAMKHANH (Khớp 100% với dxdiag và các bài tập trước)
- **Kết quả đo đặc thực nghiệm:**
 - **Sustainable Throughput (có Gaussian Think-Time):** 22.82 req/s (~1,369 requests/phút)
 - **Burst / Peak Capacity (chế độ không Think-time):** ~120 – 187 requests/sec (Spike đạt 187.67 RPS)
 - **p95 Response Time:** 8.0 ms (tối đa < 40 ms)
 - **Tỷ lệ Lỗi (Error Rate):** 0.00% (16,394/16,394 requests thành công 100%)
 - **Trần Bộ nhớ (Memory Ceiling):** RAM tiến trình node.exe duy trì ổn định ở mức ~59–86 MB.

2.5. Số Lượng Bug & Vấn Đề Hiệu Năng Phát Hiện (11 Bugs / Issues)

Đã phát hiện và báo cáo đầy đủ **11 Bugs thực tế** trên hệ thống SUT và đã được đồng bộ lên **GitHub Issues**:

Issue #	Mã Bug	Loại Bug	Mô tả Tóm tắt	Endpoint bị ảnh hưởng
#1	BUG-CONCUR-01	Concurrency	Race Condition khi kiểm tra tồn kho lúc nhiều user đặt hàng đồng thời	POST /api/checkout
#2	BUG-CONCUR-02	Concurrency	Thiếu Rollback Transaction khi tạo đơn hàng gặp lỗi	POST /api/checkout
#3	BUG-CONCUR-03	Concurrency	Xung đột trạng thái giỏ hàng trong bộ nhớ userCarts dưới tải cao	POST /api/cart
#4	BUG-CONCUR-04	Concurrency	Cơ chế khóa tài khoản (locked_until) bị nghẽn khi đăng nhập đồng thời	POST /api/login

#5	BUG- PERF-01	Performance	Quét toàn bộ bảng (<code>Table Scan</code>) do câu lệnh <code>LIKE '%...%'</code> không có Index	GET <code>/api/products?search</code>
#6	BUG- PERF-02	Performance	Xung đột khóa ghi độc quyền (<code>Exclusive Write Lock</code>) trên CSDL SQLite	POST <code>/api/checkout</code>
#7	BUG- PERF-03	Performance	Thiếu phân trang khiến trả về toàn bộ danh mục gây ngẽn bộ nhớ	GET <code>/api/products</code>
#8	BUG- PERF-04	Performance	Rò rỉ bộ nhớ (<code>Memory Leak</code>) ở đối tượng toàn cục <code>userCarts</code> sau 12 phút	Node.js Backend Engine
#9	BUG-SEC- 01	Security	Hardcoded JWT Secret Key (<code>super_secret_key_that_should_not_be_here</code>)	POST <code>/api/login</code>
#10	BUG- LOGIC-01	Business Logic	Cho phép chuyển trạng thái đơn hàng bất hợp lệ từ <code>canceled</code> sang <code>delivered</code>	PUT <code>/api/orders/:id/status</code>
#11	BUG-SEC- 02	Security / Concurrency	Sử dụng mã giảm giá đồng thời vượt quá giới hạn cho phép	POST <code>/api/checkout</code>

2.6. Liên Kết Video Demo YouTube

- **Link Video Demo Tổng Thể HW05 (>= 6 phút):** <https://youtu.be/z5PPt3clPLY>
- **Link Video Demo Agent Skill Tự Động Hóa:** <https://youtu.be/cxdNTWo8-mE>
- **Nội dung hiển thị:** Đồng khung hình giao diện JMeter chạy test và Resource Monitor (Task Manager tiến trình `node.exe`), có giọng thuyết minh tiếng Việt của sinh viên.

3. Cấu Trúc Thư Mục Bài Nộp (Folder Hierarchy)

```
HW05/  
├── submissions/  
│   └── 23127205/  
│       ├── README.md  
│       ├── git-commit-log.txt  
│       ├── data/  
│       │   ├── credentials.csv  
│       │   ├── products.csv  
│       │   └── orders.csv  
│       ├── test-plans/  
│       │   ├── 23127205_Load_20260829.jmx  
│       │   ├── 23127205_Stress_20260829.jmx  
│       │   ├── 23127205_Spike_20260829.jmx  
│       │   └── 23127205_Endurance_20260829.jmx  
│       ├── results/  
│       │   ├── load/  
│       │   ├── stress/  
│       │   └── spike/  
│       └── # Bảng tự đánh giá & Báo cáo tóm tắt  
│           # Lịch sử commit Git chi tiết  
│           # Bộ dữ liệu kiểm thử độc lập  
│           # 50 tài khoản test thực tế  
│           # Danh mục từ khóa & ID sản phẩm  
│           # Dữ liệu thanh toán & đơn hàng  
│           # 4 Test Plan JMeter (.jmx) chuẩn quy ước  
│           # Standard TG | Summary Report  
│           # Stepping TG | Aggregate Report  
│           # Ultimate TG | View Results Tree  
│           # Standard TG | 12 phút Soak Test  
│           # Log thô .jtl & HTML Dashboards  
│           # raw.jtl + HTML Dashboard  
│           # raw.jtl + HTML Dashboard  
│           # raw.jtl + HTML Dashboard
```

		└─ endurance/	# raw.jtl + HTML Dashboard
		└─ evidence/	# Minh chứng thực nghiệm
		└─ hardware/	# dxdiag png, spec txt/json (Hostname:
LAMKHANH)		└─ screenshots/	# 4 ảnh chụp JMeter + Task Manager
node.exe		└─ bugs/	# 11 ảnh chụp UI lỗi thực tế + json
evidence		└─ report/	# 7 cặp tài liệu Markdown + PDF chuẩn A4
		└─ main-report.md & .pdf	# Báo cáo tổng thể kiểm thử hiệu năng
		└─ ai-critique.md & .pdf	# Phê bình năng lực AI (200-300 từ)
		└─ ai-audit-report.md & .pdf	# Báo cáo kiểm toán sử dụng AI (14 logs)
		└─ task2-ai-analysis.md & .pdf	# Task 2: Săn lỗi diễn giải sai của AI
		└─ task3-continuous-performance-testing.md & .pdf	# Task 3: Đề xuất Continuous
Testing		└─ workflow-description.md & .pdf	# Đặc tả workflow 5 APIs & Endpoint
mapping		└─ bug-report.md & .pdf	# Báo cáo 11 lỗi thực nghiệm trên GitHub
		└─ .agents/	# AGENT SKILL TỰ ĐỘNG HÓA TÁI SỬ DỤNG (10
ĐIỂM)		└─ skills/	
		└─ performance-testing-agent/	
		└─ SKILL.md	# Đặc tả Agent Skill
		└─ scripts/	# 19 Python Automation Scripts
		└─ seed_test_accounts.py	# Tự động seed 50 tài khoản vào SQLite
		└─ setup_jmeter.py	# Tự động tải JMeter + Plugins + JVM Heap
		└─ smoke_test_sut.py	# Tự động kiểm tra tính toàn vẹn 5 API
		└─ run_jmeter.py	# CLI Runner chạy JMeter tối ưu trên
Windows		└─ reset_lockout.py	# Mở khóa tài khoản SQLite
		└─ jmx_generator.py	# Tool sinh Test Plan .jmx tự động
		└─ jtl_parser.py	# Trích xuất Ground Truth metrics từ .jtl
		└─ audit_logger.py	# Tự động ghi nhật ký AI Audit
		└─ generate_pdfs.py	# Pipeline xuất 7 file PDF chuẩn A4
		└─ ...	# Các công cụ kiểm thử & đồng bộ GitHub
		└─ eshop-sut/	# Mã nguồn SUT (Node.js/SQLite)

4. Hướng Dẫn Khởi Chạy & Tái Hiện (Quick Start)

Bước 1: Khởi động Backend SUT

```
cd eshop-sut/backend
npm install
node server.js
```

(Server backend sẽ lắng nghe tại `http://localhost:3000`)

Bước 2: Seed dữ liệu 50 tài khoản test

```
python .agents/skills/performance-testing-agent/scripts/seed_test_accounts.py
```

Bước 3: Smoke test kiểm tra 5 API endpoints

```
python .agents/skills/performance-testing-agent/scripts/smoke_test_sut.py
```

Bước 4: Chạy kiểm thử hiệu năng với JMeter

1. Chạy Load Test

```
python .agents/skills/performance-testing-agent/scripts/run_jmeter.py -n -t  
submissions/23127205/test-plans/23127205_Load_20260829.jmx -l  
submissions/23127205/results/load/raw.jtl -e -o submissions/23127205/results/load/html-report
```

2. Chạy Stress Test

```
python .agents/skills/performance-testing-agent/scripts/run_jmeter.py -n -t  
submissions/23127205/test-plans/23127205_Stress_20260829.jmx -l  
submissions/23127205/results/stress/raw.jtl -e -o submissions/23127205/results/stress/html-report
```

3. Chạy Spike Test

```
python .agents/skills/performance-testing-agent/scripts/run_jmeter.py -n -t  
submissions/23127205/test-plans/23127205_Spike_20260829.jmx -l  
submissions/23127205/results/spike/raw.jtl -e -o submissions/23127205/results/spike/html-report
```